

NYC Taxi Ride Duration Prediction

Carlo Achille Fiammenghi, Manuel Tagliaferri

Abstract

In this report, we broke down the entire data pipeline we built to predict taxi ride durations in New York City. We started with a huge dataset from the NYC Taxi and Limousine Commission—over 1.4 million trips—and put it through a full workflow: from deep exploratory data analysis (EDA) and cleaning to creating smart features based on time and geography.

To handle the messy, non-linear reality of urban traffic, we chose an XGBoost regressor. After some serious outlier removal and data transformation, the model performed well, hitting an RMSLE of 0.325 and an average error (MAE) of 174.53 seconds on our validation sets. Overall, these results show that the model is solid and can handle the complexity of city transit.

1. Introduction and Study Objectives

Predicting exactly how long a ride takes in a place as chaotic as NYC is a ‘high-level’ multi-variable regression problem. Urban traffic isn't just busy; it's inherently non-linear and wildly volatile, swinging based on the time of day, the day of the week, weather shifts, and those specific micro-geographical bottlenecks that everyone hates.

Our main goal with this project is to build a solid Machine Learning model that can predict the `trip_duration` variable using a set of basic spatial and temporal inputs, specifically pickup/drop-off coordinates and timestamps.

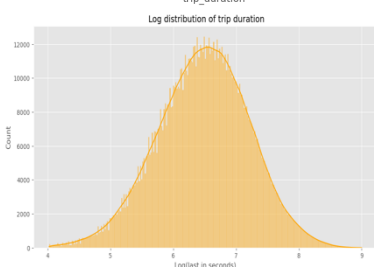
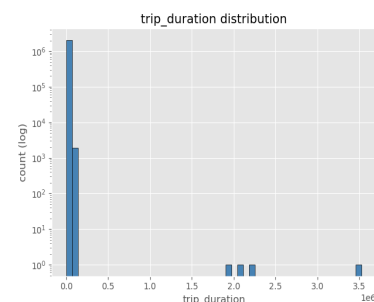
From a mathematical standpoint, we need to look at the topological nature of our data. After post-processing, the multidimensional feature space representing our dataset is strictly closed and bounded within a specific geographic and temporal range. Essentially, our operational dataset forms a **compact set**. This property is key: it guarantees that our continuous predictive functions remain bounded and reach their extrema within the analytical space we've defined.

2. Dataset Overview and First Look

We started with two main files: `train.csv` (about 1.45 million trips) and `test.csv` (around 625,000 trips). The data gives us a good mix of info to work with:

- **Time details:** Pickup and drop-off timestamps.
- **Location data:** Exact coordinates (latitude and longitude) for both the start and end of each trip.
- **Trip info:** Things like the vendor ID, the number of passengers, and the 'store and forward' flag.
- **The Goal:** Our target variable is `trip_duration`, measured in seconds.

When we did our first check using pandas (`.info()` and `.isnull().sum()`), we got some great news: there were **zero missing values (NaNs)** in the entire dataset. This was a huge relief because it meant we didn't have to waste time filling in gaps or guessing missing data using complex imputation tricks.



3. Exploratory Data Analysis (EDA)

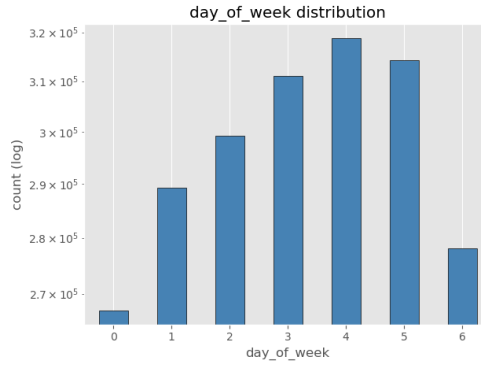
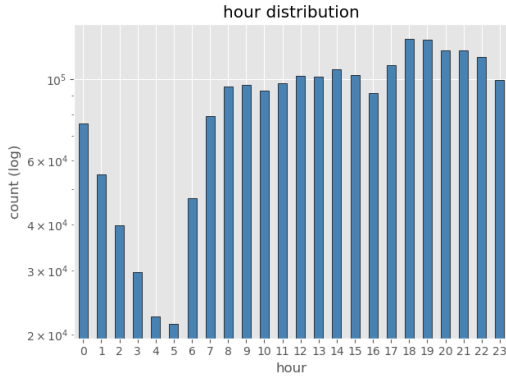
A comprehensive univariate and bivariate analysis were conducted to decode the underlying distributions, identify anomalies, and formulate hypotheses for feature engineering.

3.1 Target Variable Analysis

When we first plotted the distribution of the trip durations, the results were surprising. As you can see in the histogram, almost all the data is against the left side, but then there are these tiny bars scattered way out to the right. Some of these points are showing trips that lasted over 3.5 million seconds. To put that in perspective, that's nearly 40 days for a single taxi ride! Since nobody stays in a cab for over a month, these are clearly not real trips. They are most likely "technical errors", maybe a driver forgot to turn off the meter at the end of a shift, or the GPS

hardware simply malfunctioned. Dealing with these massive outliers was our first big step in cleaning the data. To mitigate the dominating effect of these outliers on gradient descent optimization, the target variable required a logarithmic transformation. By applying $\log(1 + x)$, the distribution was compressed into a near-Gaussian bell curve. This transformation is theoretically sound for duration prediction, as it penalizes relative errors rather than absolute errors (e.g., an error of 5 minutes is critical for a 10-minute ride, but negligible for a 2-hour ride).

3.2 Temporal Dynamics



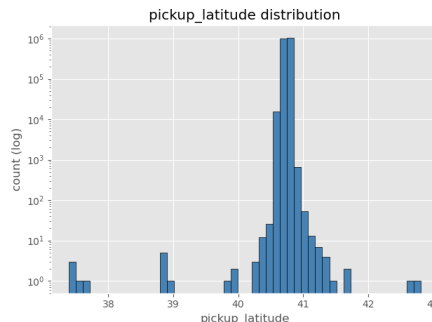
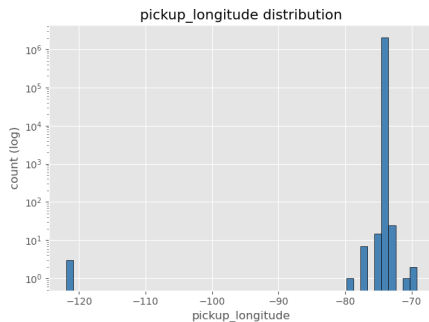
To better understand the data, we broke down the `pickup_datetime` variable into hours, days of the week, and months.

Hourly Distribution: The data shows two clear peaks, which is typical for commuting behavior. Taxi activity is at its lowest point around 4:00 AM, then steadily increases throughout the morning. After a slight plateau during the

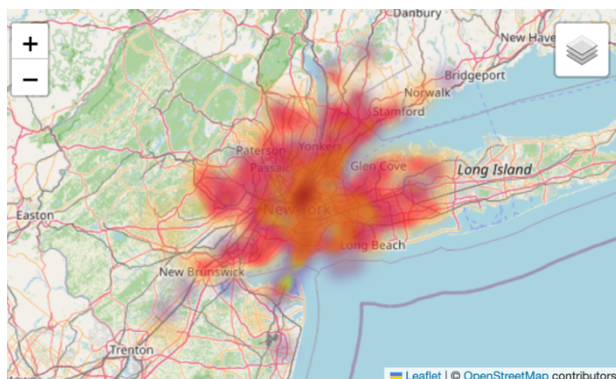
day, it reaches its absolute peak between 6:00 PM and 7:00 PM during the evening rush hour.

Weekly Distribution: Ride frequency generally increases from Monday through Friday. Activity peaks at the start of the weekend (Friday and Saturday) and then shows a sharp decline on Sunday.

3.3 Spatial Distributions and Outliers



When we plotted the raw geographic coordinates, we found several major errors (outliers). For example, some pickup and drop-off locations were listed in the middle of the Pacific Ocean or in areas very far from New York State. These are clearly data entry errors or GPS glitches.



By using Folium heatmaps to visualize the valid coordinates, we confirmed that many taxi rides are heavily concentrated within Manhattan, with a lower density of trips in the surrounding boroughs.

Visualizing the valid coordinates via Folium heatmaps confirmed that most valid taxi rides are highly concentrated within the island of Manhattan, with secondary density clusters located at major transit hubs such as JFK and LaGuardia airports.

4. Data Pre-processing and Outlier Handling

Based on what we observed during the Exploratory Data Analysis (EDA), we established specific cleaning criteria to make our model more robust. Training an algorithm on "physically impossible" data would significantly decrease its accuracy.

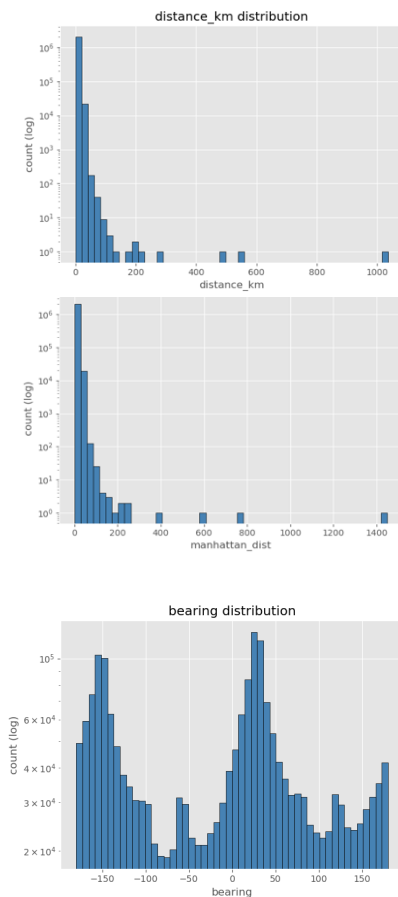
- **Geographical Bounding Box:** We defined a specific geographical area for New York City (a "bounding box"). We only kept records with latitudes between 40.15 and 41.30 and longitudes between -75.50 and -73.30. This operation effectively removed the GPS glitches mentioned earlier while preserving all the actual city traffic data.
- **Duration Clipping:** To fix errors related to the taxi meter, we applied minimum and maximum limits to the `trip_duration`. We removed rides shorter than 33 seconds, assuming a real taxi trip cannot realistically end that quickly. Similarly, we discarded trips longer than 6 hours, treating them as hardware errors or instances where the driver forgot to stop the meter. This phase only removed 0.56% of the total data but greatly improved the model's stability.
- **Speed Boundaries:** As a final filter, we calculated the average speed for every ride. We removed trips that implied a speed of 0 km/h or over 130 km/h. This ensured that our data remained consistent with the real-world physical limits of driving in an urban environment.

5. Feature Engineering

A fundamental rule in Data Analytics is that a model is only as good as the data it receives. While raw coordinates (latitude and longitude) show specific points in space, they do not explain the relationship between those points. To provide the model with better context, we created three main categories of features.

5.1 Distance Metrics

In a city like New York, which is built on a strict grid, a simple straight line is not a good way to measure travel time. Therefore, we implemented two different distance calculations:



Haversine Distance: This calculates the shortest distance between two points on a sphere (the Earth). While this is a standard mathematical approach, it is limited because it does not account for the city's actual street layout and buildings.

Manhattan Distance: This metric (also known as the L1 norm) is much more effective for New York City. It calculates the distance by following a grid pattern, similar to how a taxi must drive along streets and avenues. We adjusted this calculation to fit NYC's specific coordinates (where 1 degree equals roughly 111 km for latitude and 84.5 km for longitude). This metric provides a very accurate approximation of the actual path a vehicle must take.

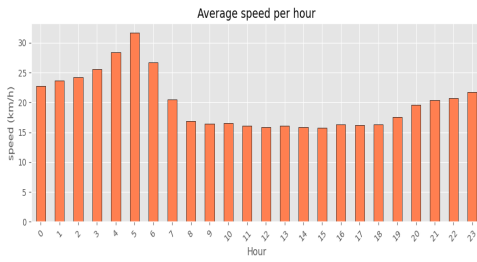
5.2 Geometric Directionality (Bearing)

To understand how the direction of travel affects ride duration, we calculated the initial compass bearing for every trip. This feature is important because traffic flow in New York City is often asymmetrical.

For example, traveling North along a wide avenue might take significantly less time than traveling East across narrower streets. This is usually due to factors like traffic light synchronization and the number of available lanes.

The "bearing distribution" graph shows a bimodal pattern (two main peaks). These peaks reflect the geographical orientation of Manhattan's grid, confirming that most trips follow the North-South or East-West structure of the city streets.

5.3 The "Golden Feature": Expected Trip Duration



Machine learning models benefit significantly from having a strong baseline or "proxy" variable to guide their predictions. To help our model, we created a new feature called **Expected Trip Duration**.

$$\text{Estimated Duration} = \left(\frac{\text{Manhattan Distance}}{\text{Average Speed for that Hour}} \right) \times 3600$$

The process involved the following steps:

1. We grouped the dataset by the **hour of the day**.
 - 1.1 We already calculated the **actual average speed (km/h)** for each specific hour.
2. We then mapped these hourly averages back into the main dataset as a new feature.
3. Finally, we calculated the estimated duration for every ride using this formula:

This "Golden Feature" is highly effective because it combines both spatial distance and traffic density into a single variable. By providing a feature that is so closely related to the actual result, we simplified the task for the predictive algorithm and improved its overall performance.

6. Predictive Modeling: XGBoost Regressor

Once the feature engineering was complete, the final step was to select the best algorithm and optimize its settings to ensure accurate predictions.

6.1 Why we chose XGBoost

We chose the **eXtreme Gradient Boosting (XGBoost)** algorithm as our predictive engine. Standard linear models often struggle with urban traffic data because the relationships are highly non-linear. For example, a 2 km trip at 4:00 AM is a completely different experience than a 2 km trip at 6:00 PM.

XGBoost excels here because it uses an "ensemble" of decision trees to map out these complex patterns. Additionally, it is naturally resistant to outliers and works well even if the data features are on different scales.

6.2 Model Settings (Hyperparameters)

To make the model as accurate as possible while avoiding "overfitting" (where a model memorizes the training data but fails on new data), we used the following parameters:

- **n_estimators = 5000**: We set a high number of boosting rounds to give the model plenty of opportunities to learn complex patterns.
- **max_depth = 9**: This allowed the decision trees to grow deep enough to capture interactions between multiple features, such as the relationship between the hour of the day and the Manhattan distance.
- **subsample = 0.7 & colsample_bytree = 0.6**: To prevent the model from becoming too reliant on just one or two "Golden Features," we forced it to train on random subsets of the data and features. This adds "noise" and makes the model more robust.
- **Regularization (alpha = 7, lambda = 1)**: These penalties were applied to simplify the model and prune unnecessary complexity, ensuring a smoother prediction surface.
- **Learning Rate Decay**: We programmed a custom callback to slowly decrease the learning rate over 5,000 epochs. This allowed the model to make large improvements early on and then "fine-tune" its accuracy as it approached the final result.
- **Objective Function**: We set the model to optimize for reg:squarederror on a **log-transformed** target $\ln p(y)$. Mathematically, this is the most effective way to optimize for the **Root Mean Squared Logarithmic Error (RMSLE)**, which was our primary metric for success.

7. Model Evaluation and Diagnostic Analysis

To evaluate our model's performance, we used an 80/20 split for training and validation. We also implemented "early stopping" during the XGBoost training process; this ensured that the model stopped learning as soon as the validation error plateaued, effectively preventing overfitting.

7.1 Quantitative Results

The final model achieved the following performance metrics on the validation set:

- **RMSLE:** 0.32520
- **Mean Absolute Error (MAE):** 174.53 seconds

With an average error of **less than 3 minutes**, the model is remarkably accurate for a system relying solely on timestamps and coordinates in a city as unpredictable as New York.

7.2 Visual Diagnostic Analysis

To understand where the model succeeds and where it reaches its limits, we analyzed the relationship between real trip durations and our predictions.

- **Actual vs. Predicted (Left Plot):** The scatter plot shows a very high density of points tightly clustered along the "Perfect Prediction" line for trips under 3,000 seconds (50 minutes). This confirms that for standard urban transit, the model is highly reliable. However, as the trips get longer, the variance increases, creating a "cone" shape. Interestingly, there is a visible vertical "wall" at exactly **7,200 seconds (2 hours)**. This suggests a specific behavior in the raw data—likely a maximum trip limit or a reporting artifact from the taxi vendors which the model tries to interpret but cannot perfectly align with.
- **Error Distribution / Residuals (Right Plot):** The residual plot shows how much our predictions deviate from the truth. For most trips, the errors are centered around the **Zero Error** line, indicating that the model is unbiased for typical rides. For exceptionally long trips, we see more points falling below the red line. This means the model tends to **under-predict** very long durations.

Conclusion from Visuals: The model is an excellent "average" predictor. It accurately captures the typical traffic flow for the vast majority of New York City trips. The errors on longer trips are expected: since the model only sees "where" and "when", it cannot account for random real-time events like a flat tire, a major traffic accident, or a road closure which are the primary causes for trips lasting much longer than the historical average.



8. Conclusion

This study successfully developed a reliable predictive pipeline for NYC taxi ride durations. By cleaning geographical outliers, implementing Manhattan distance metrics, and creating a custom time-velocity index, we transformed unstructured raw data into a highly effective feature space.

The use of an XGBoost regressor optimized with specific training techniques like learning rate decay, it proved to be very effective at modeling the complex nature of urban traffic. Ultimately, the model reached a high level of accuracy, predicting ride times with an average error of less than three minutes.

For future versions of this project, the model could be further improved by including external datasets, such as weather information (since rain or snow strongly impacts speed) or real-time traffic reports. This would allow the system to move beyond historical patterns and gain a better awareness of current road conditions.